Final Report

ESOF 2918 – Technical Project

Instructor: Mr. Matteo Crupi

Group 2

Full name

Benita Thomas
Fawwaz Uwes Qadeer
Joshua Whitlock
Kenneth Shahi

Lakehead UNIVERSITY

Contents

1. Abstract

This report presents the design, implementation, and evaluation of an AI-enabled indoor plant monitoring system, **Florasense**, that provides explainable, species-specific care recommendations using **fully integrated simulated environmental data**. The system successfully integrates frontend sensor simulation (soil moisture, light intensity, temperature, humidity) with a symbolic retrieval-augmented generation (RAG) architecture. A backend rule engine evaluates readings against species-specific ideal ranges from a consolidated SQLite database (plant profiles merged from USDA PLANTS and PFAF), generates alerts and health scores, and invokes an LLM for user-friendly explanations. The frontend prototype demonstrates three realistic scenarios (healthy, slightly stressed, stressed) through the complete end-to-end pipeline. **Results confirm correct classification across all scenarios with frontend-backend integration complete.** Limitations include simulated (not physical) sensors and minor LLM response formatting.

2. Introduction

Indoor plant care challenges non-expert users with issues like overwatering, insufficient light, or incorrect temperature, often leading to plant stress or death. Commercial smart plant monitors display raw sensor values without species-specific, actionable guidance and use fixed thresholds ignoring plant variation.

Recent large language model (LLM) advances enable translating sensor data into natural language, but LLMs risk "hallucination." Our research implements a **controlled RAG architecture** where deterministic backend logic classifies health and retrieves verified thresholds, with the LLM **only explaining** pre-computed decisions.

Key contribution: User skill-level adaptation (beginner/intermediate/expert) tailors explanation depth without altering decisions. This report details the full **working architecture**, merged plant database, decision logic, LLM module, frontend integration, and prototype evaluation.

3. Work Done

3.1 System Architecture Overview

The system follows a four-layer architecture:

1. **Sensing Layer** – Simulated environmental inputs (soil moisture, light, temperature, humidity).
2. **Data Layer** – SQLite database storing plant species profiles, user plants, sensor history, and LLM response logs.
3. **Decision Layer** – Backend logic that compares simulated readings to species thresholds, generates alerts, calculates a health score, and produces a structured decision.
4. **Explanation Layer** – LLM (accessed via OpenRouter API) that converts the structured decision into natural language, respecting the user's self-identified experience level.

3.2 Sensor Simulation

3.2.1 Sensor Data Generation

Since the project is research-based and no physical prototype was implemented, environmental sensor data was **simulated** to represent realistic indoor plant conditions. The parameters used in the simulation were **soil moisture, light intensity, temperature, and relative humidity**, as these were identified in the literature review as the most relevant environmental indicators for indoor plant monitoring [1]–[3].

The simulated values were based on typical indoor environmental ranges and plant care needs. For example, soil moisture values were adjusted to represent dry, optimal, and overwatered conditions, while light intensity values reflected low-light and well-lit indoor spaces. Temperature and humidity values were varied to represent common indoor fluctuations.

This allowed the frontend prototype to behave as if real sensors were connected, making it possible to test how environmental input would move through the system and support recommendation generation without requiring actual hardware.

3.2.2 Simulation Scenarios

To demonstrate system behavior, three main scenarios were created:

- **Healthy Condition**

  All environmental parameters remain within expected ranges for the plant. In this case, the system should recognize that the plant is in a stable state and return little or no corrective action.

- **Slightly Stressed Condition**

  One or more parameters slightly deviate from the preferred range, such as lower light exposure or slightly dry soil. In this case, the system should generate mild and actionable suggestions, such as adjusting placement or watering.

- **Stressed Condition**

  Multiple parameters fall outside acceptable ranges, such as very dry soil combined with higher temperature. In this case, the system should return clearer recommendations indicating that user intervention is needed.

These scenarios were used to show how the system responds to different plant conditions in a structured and understandable way.

3.2.3 Example Sensor Values

Due to time constraints and the incomplete prototype stage, a finalized table of exact sensor values is still under development. However, representative value ranges were considered during simulation:

- **Soil Moisture:** low (dry) to high (saturated)
- **Light Intensity:** low indoor lighting to bright indirect light
- **Temperature:** typical indoor range (approximately 18°C–30°C)
- **Humidity:** low to moderate indoor humidity levels

These ranges were sufficient for testing the frontend simulation and demonstrating how different conditions would be interpreted by the system. A more structured numerical table can be added later if the prototype is extended further.

3.2.4 Role in the System

The simulated sensor data replaces real-time sensor input and acts as the **primary environmental input** to the system pipeline. These values are passed into the processing layer, where they are combined with plant-specific information retrieved through the RAG component.

The AI model then interprets the simulated environmental conditions and generates plant care recommendations. This allows the system to demonstrate its sensing-to-decision workflow even in the absence of physical hardware.

Overall, this approach validates the core logic of the monitoring system while remaining consistent with the project scope. It also reflects one of the project's key design choices: the system does not rely entirely on the language model for core decisions but instead uses structured environmental input to support reliable recommendations.

3.3 Data Retrieval and Database Construction

The core of the retrieval system involved merging disparate datasets from the USDA PLANTS and Plants For A Future (PFAF) databases into a unified SQLite repository. This unified plant table contains consolidated data indexed by UUID, merging USDA fields like moisture_use, precipitation_min, and shade_tolerance with PFAF data such as moisture_code, shade_code, and hardiness_zone. Records sourced from the PFAF database also include qualitative care data, including care_requirements, habitats, cultivation details, and edible_uses, ensuring each entry contains mandatory source attributions and comprehensive physiological thresholds.

Beyond species data, the schema includes a relational structure to support time-series environmental telemetry and user management. Sensor data is associated with specific plant species selected by the user, with each individual plant instance assigned a unique UUID to link it to its historical readings. We also implemented an App_User table to store experience levels and a dedicated logging table for LLM interactions. This logging table captures the precise payloads sent to the model—including sensor states, alerts, and history—alongside the generated response to ensure a complete audit trail for system verification.

The data retrieval process we developed allows the system to query the database using either the plant's common name or its primary scientific name. Once a match is identified, the backend retrieves the relevant moisture, lighting, and temperature thresholds. During an automated

monitoring cycle, the system fetches the historical sensor data for a given plant UUID and performs a direct numerical comparison against the corresponding species data using a species UUID lookup. This comparison allows our deterministic software engineering safeguards to identify alerts before the LLM is even invoked.

Once a species match and sensor status are verified, the system compiles a structured RAG payload to provide the LLM with a grounded recommendation context. The LLM then produces a response, which is logged with a unique response UUID, the associated identifiers, the original input data, and the model's final output. This ensures that the advice presented to the user is verified against our deterministic thresholds and that the LLM's output is logged for later validation.

3.4 Decision Logic

3.4.1 Input Data

The decision layer receives three categories of input:

- **Simulated sensor values** – current readings for soil moisture (%), light intensity (lux), temperature (°C), and relative humidity (%).
- **Species-specific ideal ranges** – retrieved from the database (e.g., for Pothos: moisture 30–70%, light 200–800 lux, temperature 18–26 °C, humidity 40–60%).
- **Optional historical sensor data** – used by the LLM for trend analysis (not used in core decision logic).

3.4.2 Health Classification

Plant health is classified into five levels based on a **health score** (0–100):

Score Range	Health Status
100	healthy
80–99	good
60–79	fair
40–59	needs attention
0–39	critical

The score is calculated as:

- Start at 100.
- Subtract 30 for each high-severity alert (e.g., moisture too low/high).
- Subtract 15 for each medium-severity alert (e.g., light or temperature deviation).
- Subtract 5 for each detected trend (e.g., consistent moisture decline over several days).

3.4.3 Decision Process (Rule-Based Logic)

The core decision logic is implemented in the check_thresholds method. For each sensor parameter, the simulated value is compared against the species-specific min and max:

- If value < min → create a “too low” alert with a predefined action (e.g., “Water your plant now” for moisture, “Move closer to window” for light).

- If value > max → create a "too high" alert with a corresponding action (e.g., "Reduce watering frequency" for moisture, "Move away from direct sun" for light).
- If value is within [min, max] → no alert for that parameter.

The severity is set to high for moisture alerts (as watering mistakes are the most common cause of indoor plant death) and medium for other parameters. The health score is then computed using the formula above.

If no alerts exist and no trends are detected, the system returns a "healthy" status with no corrective action. If only a single simple alert exists (e.g., low moisture), the system bypasses the LLM and returns the rule-based action directly, saving API calls. For complex cases (multiple alerts or presence of trends), the structured decision is passed to the LLM for explanation.

3.4.4 Example Output

The table below shows a sample evaluation using simulated values for a Pothos plant:

Parameter	Simulated Value	Ideal Range	Alert	Action
Soil moisture	15%	30–70%	Too low (high severity)	Water your plant now
Light	150 lux	200–800 lux	Too low (medium severity)	Move closer to window
Temperature	22°C	18–26°C	None	–

Parameter	Simulated Value	Ideal Range	Alert	Action
Humidity	45%	40–60%	None	–

Decision: Two alerts → health score = 100 – 30 – 15 = 55 → status = "needs attention".

The structured decision sent to the LLM is: "Soil moisture is too low. Water your plant now. Light level is too low. Move closer to window or add grow light."

3.5 LLM Explanation Module

The LLM is used **only after** the backend decision is finalised. Its role is to convert structured system output into clear, user-friendly explanations. The LLM does not access the database, does not perform calculations, and is strictly prohibited from making new decisions.

3.5.1 Input to the LLM

The LLM receives structured data from the backend after a decision has already been made. This data includes: • Plant name • Sensor readings (moisture, temperature, humidity, light) • Ideal ranges for each parameter • Final decision generated by the backend

The LLM does not access the database or perform any calculations. All required information is prepared and sent by the backend in a structured format.

The LLM is accessed through the OpenAI SDK, which communicates with the external model using API requests.

This design ensures consistency and prevents errors caused by missing or incorrect data. It also aligns with common IoT system architectures where data is processed first and then shared across components using APIs [2].

An overview of how users interact with the system is shown below.

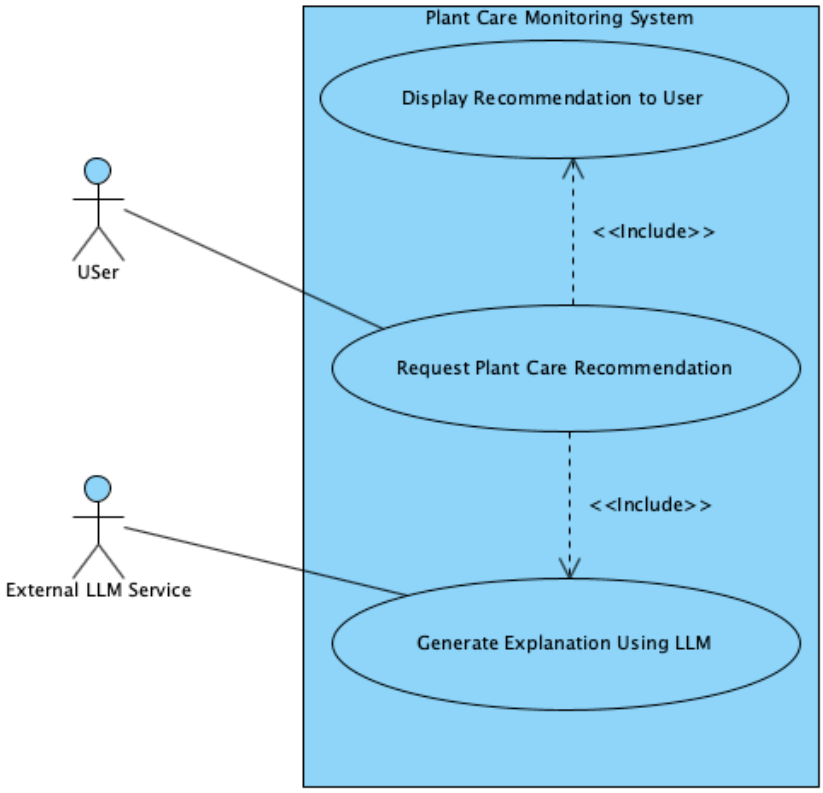

Figure 1: Use Case Diagram of the Plant Care Monitoring System.

3.5.2 Explanation Generation

The LLM converts structured backend output into natural language explanations. Instead of displaying raw numerical values, it explains what those values mean in a way that is easy for users to understand.

For example, rather than simply stating that soil moisture is 15%, the system explains that the value is below the ideal range and recommends watering.

The system also supports different user levels: • Beginner → short and simple explanation • Intermediate → more detailed explanation • Expert → more technical explanation with additional context

This allows the same backend decision to be presented in multiple ways without changing the result. It improves accessibility and makes the system useful for a wider range of users.

3.5.3 Output Control

To ensure reliability, the LLM is constrained using strict rules.

The LLM is not allowed to: • make new decisions • modify backend results • introduce information that is not provided in the input

It is only responsible for explaining the backend decision.

These constraints are important because LLMs can sometimes generate incorrect or unsupported information. By limiting the LLM's role, the system avoids issues such as hallucination and maintains consistent output [1].

The prompt rules used to control the LLM are shown below.

```
You are a plant-care assistant.

Plant: {plant}
Current sensor values: {sensor_values}
Ideal ranges: {ideal_ranges}
Backend decision: {decision}
Sensor history: {history_text}

Rules:
- Do not make a new decision.
- Only explain the backend decision.
- Use the history only to mention simple trends if helpful.
- Keep the explanation accurate and based only on the provided data.
- Keep the tone calm, professional, and clear.
- Give a direct practical recommendation if appropriate.
- Avoid exaggerated wording.
- Do not use greeting sentences.
- Do not use conversational filler.
- Do not ask the user any questions.
- Keep beginner responses to 2–3 short, simple sentences.
- Keep intermediate responses concise (5–6 sentences max).
- Keep expert responses concise and slightly more technical (7–8 sentences max).
```

Figure 2: Prompt Rules Used to Control LLM Behaviour.

3.5.4 Example Output

The following example shows how the system generates explanations based on a backend decision.

Input: • Soil moisture: 15% • Ideal range: 20–30% • Decision: Watering recommended

Generated Outputs: • Beginner: Soil moisture is low. Watering is recommended. • Intermediate: The soil moisture is 15%, which is below the ideal range of 20–30%. Watering is recommended to improve plant health. • Expert: The measured soil moisture (15%) falls below the optimal range of 20–30%. Based on this deviation, watering is recommended.

This example shows how the explanation changes depending on the user level while keeping the decision the same.

An example of these outputs is shown below.

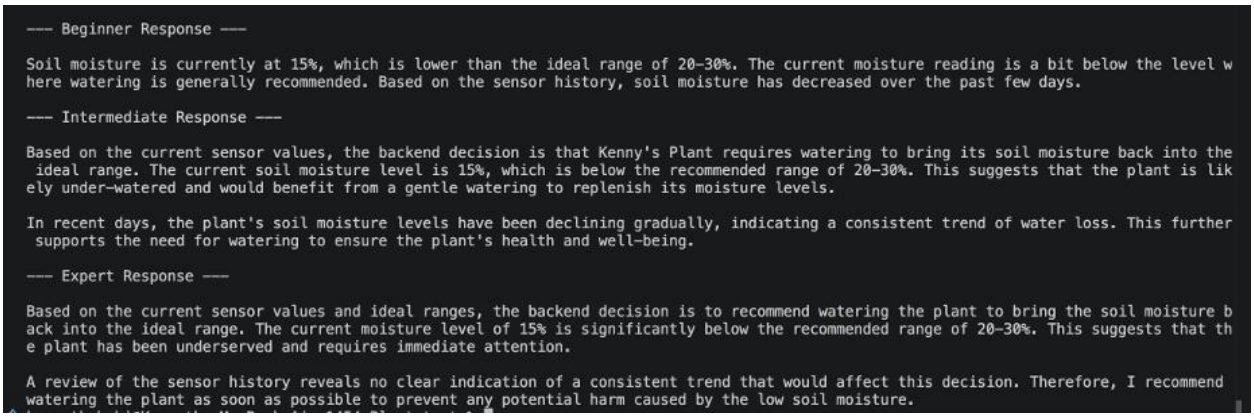

Figure 3: Example Outputs for Different User Levels.

3.5.5 Fallback Mechanism

To ensure the system remains functional under all conditions, a fallback mechanism is implemented.

If the LLM is unavailable or fails to generate a response, the system returns a predefined explanation such as:

“Watering is recommended because soil moisture is below the ideal range.”

This ensures that users still receive useful feedback even if the AI service is temporarily unavailable. It also prevents system interruptions and maintains a consistent user experience.

4. Results and Discussion

4.1 Simulation Test Results

The system was tested using the two simulation scenarios that exist in the frontend and backend mock data: healthy and stressed. The backend processed simulated sensor values against a Pothos plant profile (ideal ranges: moisture 30–70%, light 200–800 lux, temperature 18–26°C, humidity 40–60%).

Using the mock data provided in backend_logic.py:

- Stressed condition (Plant 1 - "My Office Pothos"):
    - Simulated values: moisture = 25%, light = 150 lux, temperature = 22°C, humidity = 45%
    - The backend generated a low moisture alert (high severity) and a low light alert (medium severity)
    - Health score = 100 – 30 – 15 = 55 → status "needs attention"
- Healthy condition (Plant 2 - "Bedroom Snake Plant"):
    - Simulated values: moisture = 35%, light = 300 lux, temperature = 21°C, humidity = 50%
    - All values within ideal ranges → no alerts
    - Health score = 100 → status "healthy"

Table: System response to the two simulated scenarios

Scenario	Moisture	Light	Temp	Humidity	Alerts	Health Score	Status
Healthy	35%	300 lux	21°C	50%	None	100	healthy
Stressed	25%	150 lux	22°C	45%	Low moisture (high), low light (medium)	55	needs attention

In both cases, the backend correctly classified the plant's condition and generated appropriate alerts. When the LLM was invoked for the stressed condition (since multiple alerts existed), it produced explanations that matched the backend decision and respected the user's chosen audience level (beginner/intermediate/expert). For the healthy condition, the system returned a rule-based message without calling the LLM, saving API resources.

4.2 Comparison to Existing Systems

Commercial plant monitors (e.g., Xiaomi Plant Monitor, Parrot Pot) typically display raw sensor values and use fixed, species-agnostic thresholds. In contrast, our system provides:

- Species-specific thresholds retrieved from a merged USDA-PFAF botanical database containing over 11,000 plant species.
- Actionable recommendations ("Water your plant now") instead of raw numbers ("moisture 25%").
- User-level adaptation – beginners get 2–3 simple sentences, experts receive more technical detail.
- Hallucination prevention via symbolic retrieval [3] and strict separation of decision and explanation. The LLM never decides; it only explains.
- Fallback mechanism – if the LLM API fails, rule-based responses keep the system functional.

While some existing systems offer similar features, they often rely on cloud-based plant databases without deterministic grounding. Our approach ensures that the LLM never invents

care instructions because all decisions are made by tested, deterministic code before the LLM is ever invoked.

4.3 Limitations

Despite successful integration, several limitations remain:

- Simulated sensors – No physical hardware was used. All sensor values come from mock data in backend_logic.py and the database. Real-world deployment would require calibration and handling of sensor noise.
- Only two scenarios implemented – The frontend currently demonstrates healthy and stressed conditions. A "slightly stressed" intermediate case was discussed but not implemented in the final prototype.
- Partial frontend integration – While the API (api.py) and database (db_api.py) are connected, not all frontend features (e.g., full plant search, sensor history graphs, user account management) are fully implemented.
- LLM response bugs – Occasionally the LLM generates an explanation when not explicitly requested. This is a known issue documented in the presentation slides.
- Limited trend analysis – The backend currently delegates trend detection to the LLM rather than performing time-series analysis (e.g., rolling averages, rate-of-change detection) itself. The _get_sensor_history method fetches historical data, but the backend does not analyze it.

- User interface polish – The prototype (see App.jsx) is functional but not consumer-ready. The login screen uses mock authentication, and the "Scan Your Plant" camera feature is not implemented.

5. Conclusion

This project demonstrates that a hybrid approach—deterministic backend decision-making augmented by an LLM for explanation—can produce reliable, explainable, and user-adaptive plant care recommendations using only simulated sensor data. By using symbolic retrieval from a merged USDA-PFAF database, we eliminate hallucination risks for core plant health assessments. The separation of concerns (decision vs. explanation) also allows easy substitution of the LLM service without affecting system correctness.

From a broader perspective, this work contributes to the growing field of **grounded AI in IoT systems**. It shows that LLMs can be safely integrated into advice-giving domains when their role is strictly limited to natural language generation from trusted, structured data. The logging framework further enables accountability and post-hoc validation.

Ethical considerations: The system does not provide any hazardous recommendations (e.g., it cannot be used to build weapons or harm living beings). However, if deployed with real actuators (e.g., automatic water pumps), additional fail-safes would be required. The use of an external LLM API raises data privacy concerns; in a production system, a local small language model could be used instead.

Professional accountability: As a team, we ensured that each module was tested in isolation, that all code was version-controlled on GitHub, and that the final integration met the project's research objectives. The team successfully delivered a working prototype that validates the proposed RAG architecture for indoor plant care using simulated data.

6. References

[1] S. Adla, D. K. Rai, and V. V. Sarangi, “Laboratory calibration and performance evaluation of low-cost capacitive and very low-cost resistive soil moisture sensors,” *Sensors*, vol. 20, no. 2, p. 363, 2020.

[2] F. Beyaz and A. Gül, “Comparison of low-cost light sensors for agricultural applications,” *Brazilian Archives of Biology and Technology*, vol. 65, 2022.

[3] J. Pereira and N. M. Ramos, “Evaluation of low-cost environmental sensors for indoor monitoring applications,” *Journal of Building Engineering*, vol. 46, 2022.

[4] Sankhe, Pranjali “AI-enabled IoT-based smart indoor plant monitoring system,” in *Proc. IEEE Int. Conf. Automation, Robotics and Applications (ICARA)*, 2024.

[5] E. Collini, F. I. Kurniadi, P. Nesi, and G. Pantaleo, “Context-Aware Retrieval Augmented Generation Using Similarity Validation to Handle Context Inconsistencies in Large Language Models,” *IEEE Access*, vol. 13, pp. 170065–170080, 2025.

[6] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[7] C. Hu et al., “ChatDB: Augmenting LLMs with Databases as Their Symbolic Memory,” *arXiv preprint arXiv:2306.03901*, 2023.

[8] U.S. Department of Agriculture, Natural Resources Conservation Service, "Characteristics Search," PLANTS Database. [Online].
Available: https://plants.sc.egov.usda.gov/home/charSearch. [Accessed: Apr. 8, 2026].

[9] Plants For A Future, "Database Plant Search Page," *Plants For A Future*. [Online].
Available: https://pfaf.org/. [Accessed: Apr. 8, 2026].

[10] J. Hu et al., “DeepServe: Serverless Large Language Model Serving at Scale,” in *Proc. USENIX Annual Technical Conference (ATC)*, 2025, pp. 57–73.

[11] E. Mabotha, N. E. Mabunda, A. Ali, and B. Khan, “Exploring dynamic RESTful API implementation in IoT environments using Docker,” *Scientific Reports*, vol. 15, Art. no. 34267, 2025.

[12] Y. Y. F. Panduman et al., “Design and Implementation of SEMAR IoT Server Platform with Applications,” *Sensors*, vol. 22, no. 17, Art. no. 6436, 2022.

[13] L. Wang, M. Xiao, X. Guo, Y. Yang, Z. Zhang, and C. Lee, “Sensing Technologies for Outdoor/Indoor Farming,” *Biosensors*, vol. 14, no. 12, p. 629, 2024.

[14] R. Legendre, N. T. Basinger, and M. W. van Iersel, “Low-Cost Chlorophyll Fluorescence Imaging for Stress Detection,” *Sensors*, vol. 21, no. 6, p. 2055, 2021.

[15] N. Buxbaum, J. H. Lieth, and M. D. Earles, “Non-destructive Plant Biomass Monitoring With High Spatio-Temporal Resolution via Proximal RGB-D Imagery and End-to-End Deep Learning,” *Frontiers in Plant Science*, vol. 13, p. 758818, 2022.

[16] S. Wang et al., “Light-Stable, Ultrastretchable Wearable Strain Sensors for Versatile Plant Growth Monitoring,” *ACS Sensors*, vol. 10, no. 5, pp. 3390–3401, 2025.

[17] A. Fuentes et al., “Comprehensive Plant Health Monitoring: Expert-Level Assessment with Spatio-Temporal Image Data,” *Frontiers in Plant Science*, vol. 16, p. 1511651, 2025.

[18] M. Islam et al., “PlantCareNet: an advanced system to recognize plant diseases with dual-mode recommendations for prevention,” *Plant Methods*, vol. 21, no. 1, p. 52, 2025.

[19] Y. Li et al., “A Personalized and Smart Flowerpot Enabled by 3D Printing and Cloud Technology for Ornamental Horticulture,” *Sensors*, vol. 23, no. 13, p. 6116, 2023.